working and fast processing layer. When implementing a caching layer, you can employ a variety of caching techniques to keep the data that's cached fresh, which we'll review later. Figure 11.1 shows that the cache sits between your application and the database.

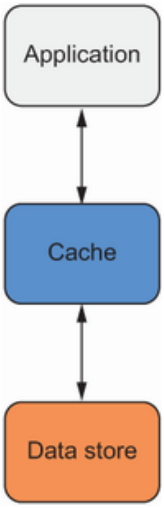

Figure 11.1 The cache sits between the application and the database.

A cache comes with the following benefits:

- Serving read traffic from the caching layer frees resources on your primary data store, for example, for write requests.
- It speeds up your application because the caching layer responds more quickly than your data store.
- It allows you to downsize your data store, which is typically more expensive than the caching layer.

Most caching layers reside in memory, which is why they are so fast. The downside is that you can lose the cached data at any time because of a hardware defect or a restart. Always keep a copy of your data in a primary data store with disk durability, like the relational database in the mobile game example. Alternatively, Redis has optional failover support. In the event of a node failure, a replica node will be elected to be the new primary and will already have a copy of the data. On top of that, some in-memory databases also support writing data to persistent storage to be able to restore the data after a reboot or outage.